

TROUBLESHOOTING DOCUMENT

Automated React Deployment Using Jenkins, Docker, AWS EC2, Prometheus and Grafana

Project 1 | DevOps Engineering

1. Project Overview

This project automates deployment of a React application using a CI/CD pipeline. Jenkins retrieves source code from GitHub, builds the React application, creates a Docker image, pushes it to Docker Hub, and deploys the container to AWS EC2. The EC2 server is monitored using Node Exporter, Prometheus, and Grafana. A Grafana CPU alert is configured with Gmail email notification.

```
Developer
  ↓
GitHub (dev branch)
  ↓
Jenkins
  ↓
React Build
  ↓
Docker Image
  ↓
Docker Hub
  ↓
AWS EC2
  ↓
React Application
  ↓
Node Exporter
  ↓
Prometheus
  ↓
Grafana
  ↓
CPU Alert
  ↓
Gmail Notification
```

2. Troubleshooting Approach

- Identify the symptom or error.
- Check logs, configuration, and service status.
- Find the root cause.
- Apply the required fix.
- Restart or rerun the affected service or pipeline.
- Verify that the issue is resolved.
- Document the solution and prevention.

3. React Build Failed in Jenkins

Symptom: The Jenkins pipeline failed during the React build stage. Jenkins was running with CI=true, which caused Create React App to treat warnings as errors.

Root Cause: The CI environment behavior caused the React build to fail because warnings were treated as build errors.

Diagnosis: The failure occurred during the Build React App stage in the Jenkins console output.

Solution: `bat 'set "CI=" && set "DISABLE_ESLINT_PLUGIN=true" && npm run build'`

Verification: The pipeline was executed again and the React application compiled successfully. The Jenkins output showed 'Compiled successfully.'

Prevention: The build command was documented in the Jenkinsfile so the same CI-related build behavior would not cause the pipeline to fail again.

Interview Explanation: Initially, my React build failed in Jenkins because Jenkins sets CI=true, and Create React App treats warnings as errors in that mode. I modified the build command to unset CI and disable the ESLint plugin for the build. After that, the React application compiled successfully.

4. React Build Failed on AWS EC2

Symptom: The React application build on EC2 failed with: Environment key "jest/globals" is unknown.

Root Cause: The installed React/ESLint dependency configuration caused an ESLint compatibility problem during the build.

Diagnosis: The installed versions were checked with `npm list eslint eslint-config-react-app`. The environment included react-scripts 5.0.1, eslint-config-react-app 7.0.1, and eslint 8.57.1.

Solution: `CI=false DISABLE_ESLINT_PLUGIN=true npm run build`

Verification: The production build completed successfully.

Prevention: The build command was used after checking dependency versions instead of forcing dependency changes.

Interview Explanation: The React build on EC2 failed because of an ESLint configuration compatibility issue. I checked the installed ESLint versions and then built the application with CI disabled and the ESLint plugin disabled for the production build. The build completed successfully.

5. Jenkins SSH Authentication Failed

Symptom: The Jenkins pipeline failed during EC2 deployment with: Permission denied (publickey,gssapi-keyex,...).

Root Cause: Jenkins was not using the correct private SSH key for connecting to the EC2 instance.

Diagnosis: The EC2 instance was accessible manually with the correct key pair, while Jenkins could not authenticate. The Jenkins credential ec2-ssh-key was checked.

Solution: The correct EC2 private key associated with the instance was configured in Jenkins. The SSH username was ec2-user and the credential ID remained ec2-ssh-key.

Verification: The Jenkins pipeline was executed again. Jenkins successfully connected to EC2, deployed the Docker container, and finished with SUCCESS.

Prevention: The EC2 key pair and Jenkins credential were checked before rerunning deployment.

Interview Explanation: The Jenkins deployment initially failed because Jenkins was using an incorrect SSH credential. I replaced the credential with the correct EC2 key pair and kept the same credential ID used in the Jenkinsfile. After that, Jenkins successfully connected to EC2 and deployed the application.

6. Docker Container Deployment Conflict

Symptom: During redeployment, an existing Docker container with the same name could prevent the new container from starting.

Root Cause: The previous Docker container could still exist on the EC2 instance.

Diagnosis: The deployment stage was reviewed for container name conflicts.

Solution: `docker stop react-container || true`
`docker rm react-container || true`
`docker run -d -p 80:80 --name react-container --restart unless-stopped akshayamanimuthu/react-app:dev`

Verification: The application became accessible through the EC2 address and the container ran with port 80 mapped to the Nginx container.

Prevention: The deployment pipeline handles an existing container before starting the new version.

Interview Explanation: To make deployment repeatable, the pipeline stops and removes the existing container before starting the new container. This prevents container-name conflicts during redeployment.

7. Node Exporter Verification

Symptom: Monitoring required system-level metrics from the EC2 server.

Root Cause: Node Exporter had to be installed, started, and expose metrics on port 9100.

Diagnosis: The service was checked with `systemctl status node_exporter` and the metrics endpoint with `curl`.

Solution: `sudo systemctl daemon-reload`
`sudo systemctl enable --now node_exporter`
`curl http://localhost:9100/metrics | head`

Verification: The service showed Active: active (running), and the metrics endpoint returned metrics.

Prevention: The service status and metrics endpoint were verified after installation.

Interview Explanation: I used Node Exporter to collect EC2 system metrics such as CPU, memory, disk and network information. I verified the service status and checked the `/metrics` endpoint to confirm that metrics were being exposed.

8. Prometheus Target Verification

Symptom: Prometheus needed to collect metrics from Node Exporter.

Root Cause: The Node Exporter target had to be configured correctly and Prometheus had to be running.

Diagnosis: The configuration was checked and validated with promtool.

Solution: The target was configured as:

```
scrape_configs:
  - job_name: "node_exporter"
    static_configs:
      - targets: ["localhost:9100"]
```

Validation command:

```
sudo /opt/prometheus/promtool check config /etc/prometheus/prometheus.yml
```

Verification: Prometheus readiness returned 'Prometheus Server is Ready.' and target health showed 'up'. The Prometheus UI also showed the Node Exporter target as UP.

Prevention: The Prometheus configuration was validated before relying on monitoring data.

Interview Explanation: Prometheus was configured to scrape Node Exporter on localhost port 9100. I validated the configuration, checked Prometheus readiness and verified that the Node Exporter target was UP.

9. Grafana SMTP Not Configured

Symptom: Grafana initially displayed 'SMTP not configured' when testing the email contact point.

Root Cause: The SMTP configuration lines in grafana.ini were commented out with a semicolon (;), so Grafana ignored them.

Diagnosis: The SMTP section was inspected and the settings were found to be commented.

Solution: [smtp]
enabled = true
host = smtp.gmail.com:587
user = YOUR_GMAIL_ADDRESS
password = YOUR_GOOGLE_APP_PASSWORD
from_address = YOUR_GMAIL_ADDRESS
from_name = Grafana
startTLS_policy = MandatoryStartTLS

Grafana was then restarted.

Verification: Grafana accepted the SMTP configuration and the email contact point test became available.

Prevention: SMTP settings must be uncommented and Grafana must be restarted after changing grafana.ini.

Interview Explanation: Grafana initially reported that SMTP was not configured. I checked grafana.ini and found that the SMTP settings were commented out. I enabled the settings, configured Gmail SMTP with an App Password, restarted Grafana and successfully tested the email notification.

10. Gmail SMTP Authentication Failed

Symptom: Grafana initially returned: 535 5.7.8 Username and Password not accepted.

Root Cause: The Gmail SMTP authentication credentials were not accepted.

Diagnosis: The Grafana SMTP test response identified the problem as an authentication failure.

Solution: A Google App Password was created and configured in Grafana instead of the normal Gmail password. The SMTP connection used smtp.gmail.com:587 with MandatoryStartTLS.

Verification: The contact point test succeeded and a test Gmail notification was received.

Prevention: SMTP credentials should be stored securely and should not be shared in documentation or screenshots.

Interview Explanation: Gmail rejected the SMTP authentication with error 535. I configured a Google App Password for Grafana and updated the SMTP password. After restarting Grafana, the contact point test succeeded and I received the email.

11. CPU Alert Testing

Symptom: The Grafana CPU alert initially remained Normal because EC2 CPU usage was not high enough to trigger it.

Root Cause: The CPU usage condition had not been reached for the required evaluation period.

Diagnosis: CPU usage was monitored from the Grafana dashboard. For testing, the alert threshold was temporarily lowered.

Solution: stress-ng --cpu 2 --timeout 5m

Verification: The Grafana alert changed to Firing, proving that the alert mechanism worked. After testing, the final configuration was restored to CPU above 80% with a 5-minute pending period.

Prevention: The alert was tested only for verification and the final project threshold was restored afterward.

Interview Explanation: I tested the Grafana alert by generating CPU load on the EC2 instance. The alert changed to Firing, which confirmed that the alerting mechanism was working. After testing, I restored the final threshold to 80% with a 5-minute pending period.

12. Final Troubleshooting Verification

Component	Final Status
GitHub repository	Working
Jenkins pipeline	SUCCESS
React build	Successful

Docker image	Built
Docker Hub image	Pushed
AWS EC2 deployment	Successful
React application	Accessible
Node Exporter	Running
Prometheus	Running
Prometheus target	UP
Grafana	Running
Monitoring dashboard	Working
High CPU alert	Tested successfully
Gmail notification	Successfully received

13. Important Troubleshooting Commands

Check Docker containers: `docker ps`

Check Docker logs: `docker logs react-container`

Check Node Exporter: `sudo systemctl status node_exporter`

Check Prometheus: `sudo systemctl status prometheus`

Check Prometheus readiness: `curl http://localhost:9090/-/ready`

Check Node Exporter metrics: `curl http://localhost:9100/metrics | head`

Check Grafana: `sudo systemctl status grafana-server`

Restart Grafana: `sudo systemctl restart grafana-server`

14. Conclusion

The major issues encountered during the project were related to React build configuration, Jenkins SSH authentication, Docker deployment, monitoring configuration, and Grafana email authentication. Each issue was diagnosed using logs, service status, configuration checks, and command-line verification. After applying the fixes, the CI/CD pipeline, AWS deployment, monitoring system, alerting mechanism, and email notification were successfully verified.

This troubleshooting process improved practical understanding of Jenkins, Docker, AWS EC2, Linux services, Prometheus, Grafana, CI/CD deployment, and production-style troubleshooting.